

LAB 2 – Flappy Bird

Il gioco interattivo che ho realizzato si ispira ad un gioco realmente esistente chiamato “**Flappy Bird**”, l’obiettivo è quello di far passare l’uccellino attraverso gli ostacoli senza toccarli. Per svilupparlo sono partito dal progetto di esempio “*LAB 2 2D DUCK*” che ho modificato sia dal punto di vista del gameplay che della grafica, per avvicinarlo il più possibile al videogioco originale. Una volta avviato è possibile far volare l’uccellino cliccando il tasto “**W**”. Quando il tasto non viene premuto, il personaggio plana lentamente verso il basso. Combinando questi due movimenti, il giocatore deve cercare di attraversare gli ostacoli posizionati nella parte superiore e inferiore dello schermo. Se si colpisce l’ostacolo in basso o in alto il gioco termina.

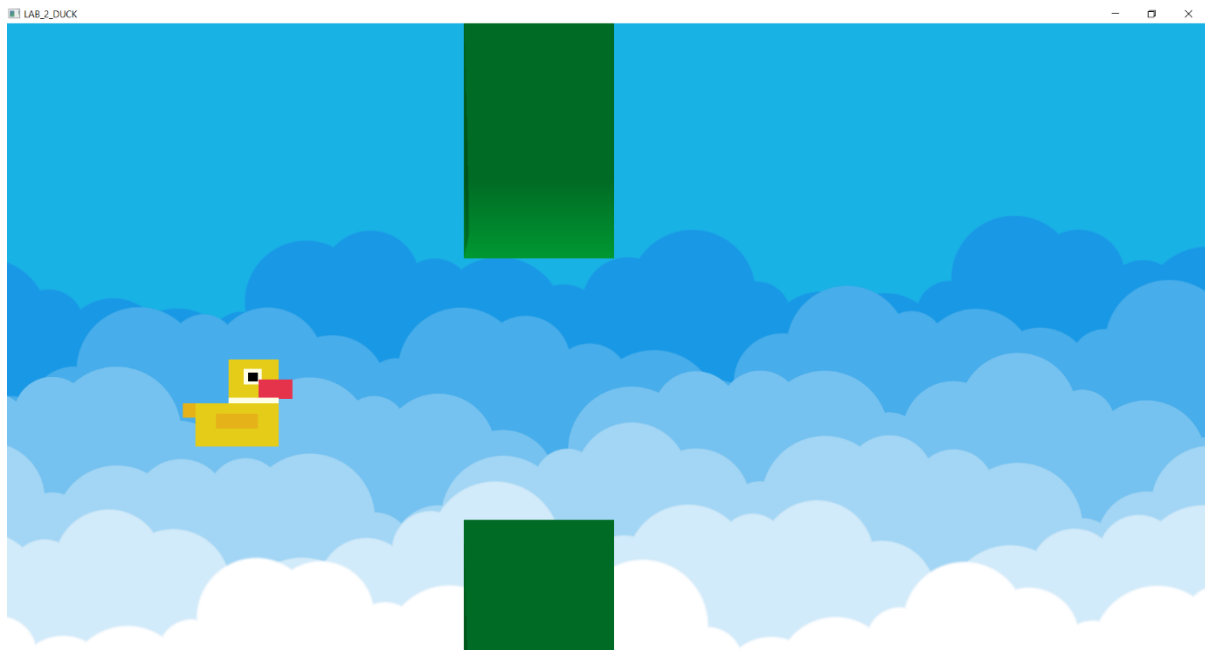


Figura 1 – Videogioco

1 Ostacoli

La prima modifica che ho effettuato è stata la rimozione degli ostacoli triangolari presenti nel progetto originale, sostituendoli con **due rettangoli verticali**: uno che parte dal basso e uno dall’alto, lasciando uno spazio centrale per permettere il passaggio dell’uccellino. Per rendere il gioco più dinamico la **posizione** degli ostacoli è **variabile**. La posizione dell’ostacolo inferiore (dyobstaclesdown) viene generata randomicamente. Quella dell’ostacolo superiore

viene calcolata aggiungendo una costante a $dyObstaclesDown$, così da mantenere sempre lo stesso spazio tra i due ostacoli (lo spazio utile per far passare l'uccellino). Una volta che l'uccellino supera un ostacolo, ne viene generato automaticamente uno nuovo, continuando il flusso di gioco.

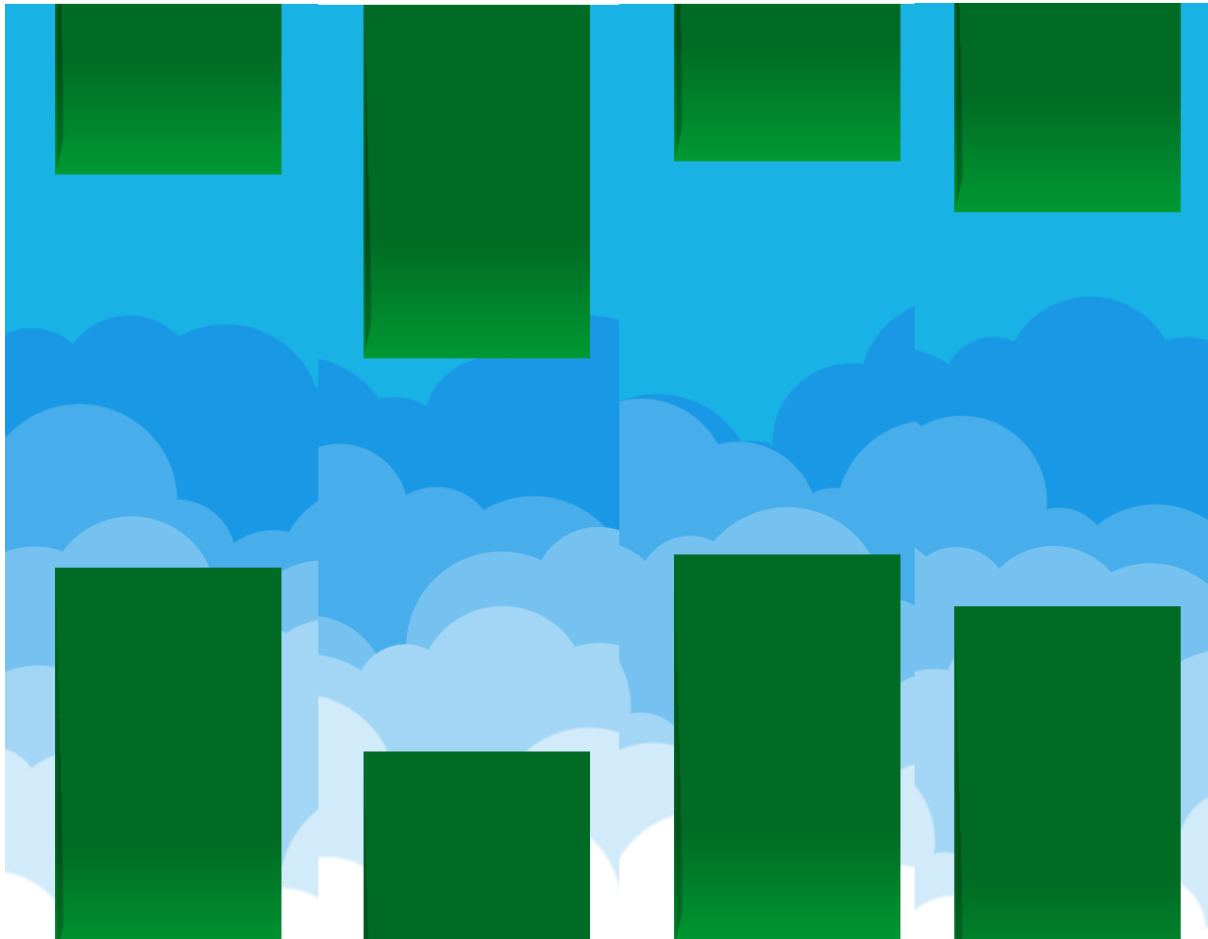
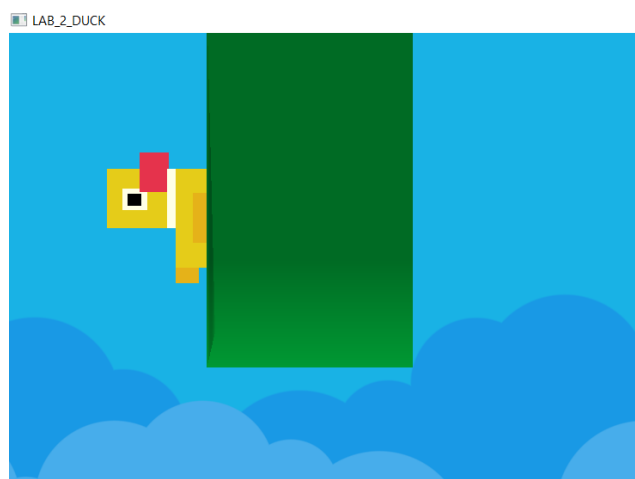


Figura 2 - Differenza nella posizione degli ostacoli

Per controllare se l'uccellino ha colpito un ostacolo si effettua un **doppio controllo**. Si verifica se la posizione X del personaggio è maggiore o uguale alla X dell'ostacolo. Se il primo controllo è vero, si controlla che la posizione verticale del personaggio non sia inferiore alla sommità dell'ostacolo inferiore né superiore alla base dell'ostacolo superiore. Poiché



l'uccellino ha un'altezza visiva, alla sua posizione Y viene aggiunta una costante per tener conto della testa. Se l'uccellino colpisce un ostacolo, il gioco termina e il character ruota di 90° come se avesse colpito il muro. Per ricominciare a giocare, è sufficiente premere il tasto “R”, che esegue il reset del gioco.

2 Movimento e sistema particellare

Quando viene premuto il tasto “W”, l'uccellino ruota di 45° a indicare graficamente che sta salendo. Premendo il tasto della tastiera la velocità verticale aumenta e quella verso il basso viene azzerata. Se invece il tasto non viene premuto, la velocità verso l'alto viene annullata mentre aumenta gradualmente quella verso il basso, simulando così il planare dell'uccellino. Il cambiamento della posizione sull'asse Y del personaggio avviene modificando direttamente il valore della variabile *posy*.

Oltre a gestire il movimento, la pressione del tasto “W” attiva anche un **sistema particellare**. Questo genera una **spirale** di particelle dietro l'uccellino, accentuando visivamente l'azione del volo. Le particelle si muovono verso sinistra, cioè nella direzione opposta rispetto a quella del personaggio, creando l'effetto di una scia. Durante il loro



Figura 3 - Sistema particellare

movimento, il colore delle particelle varia gradualmente dall'azzurro al verde, mentre il valore di alpha diminuisce progressivamente, rendendole sempre più trasparenti fino a scomparire.

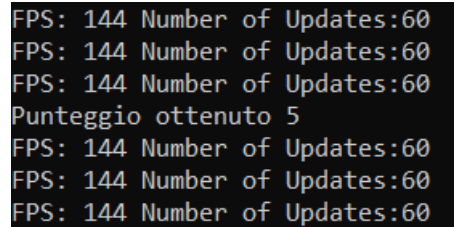
3 Shader e parti grafiche

Per migliorare l'aspetto visivo del gioco, ho apportato alcune modifiche alla grafica del personaggio principale. In particolare, ho cambiato il **colore** dell'uccellino, rendendolo giallo per richiamare l'aspetto del protagonista originale di *Flappy Bird*, e ho leggermente allargato il becco. Ho integrato anche due **shader personalizzati**. Il primo, identificato come

programIdObstacles, viene utilizzato per colorare gli ostacoli di verde, applicando un gradiente che dona maggiore profondità evitando un effetto piatto. Il secondo, *programIdClouds*, utilizza uno shader del sito **glsandbox** che inserisce sullo sfondo una serie di nuvole in movimento.

4 Punteggio

Il sistema di **punteggio** è molto semplice. Ogni volta che l'uccellino riesce a superare un ostacolo senza collisioni, viene incrementata la variabile *points*, che tiene traccia del punteggio totale ottenuto dal giocatore durante la sessione di gioco. Quando l'uccellino colpisce un ostacolo e quindi il giocatore perde nella **console** viene stampato il punteggio.

A screenshot of a console window with a black background and white text. It displays game performance metrics and the current score. The text is as follows:

```
FPS: 144 Number of Updates:60
FPS: 144 Number of Updates:60
FPS: 144 Number of Updates:60
Punteggio ottenuto 5
FPS: 144 Number of Updates:60
FPS: 144 Number of Updates:60
FPS: 144 Number of Updates:60
```

Figura 4 - Punteggio